Typage et reduction

Noyau OCaml

Types

$$\begin{array}{rll} \tau ::= & \mathsf{num} & \text{(type des nombres)} \\ & \mathsf{str} & \text{(type des chaînes)} \end{array}$$

Expressions

$$\begin{array}{rll} e ::= & x & \text{(variable)} \\ & n & \text{(nombre littéral)} \\ & s & \text{(chaîne littérale)} \\ & e_1 + e_2 & \text{(addition)} \\ & e_1 \mathbin{\hat{}} e_2 & \text{(concaténation)} \\ & \texttt{let } x = e_1 \texttt{ in } e_2 & \text{(définition)} \end{array}$$

Noyau OCaml : réduction

$$\frac{e_1 \to e_1'}{e_1 + e_2 \to e_1' + e_2} \qquad \frac{e_2 \to e_2'}{n + e_2 \to n + e_2'} \qquad \frac{n_1 + n_2 = n}{n_1 + n_2 \to n}$$

$$\frac{e_1 \to e_1'}{\texttt{let } x\texttt{=}e_1 \texttt{ in } e_2 \to \texttt{let } x\texttt{=}e_1' \texttt{ in } e_2} \qquad \frac{}{\texttt{let } x\texttt{=}v \texttt{ in } e_2 \to e_2\{v/x\}}$$

Règles pour la concaténation $e_1 \hat{} e_2$: idem que $+$.

Noyau OCaml : réduction

$$\frac{e_1 \to e_1'}{e_1 + e_2 \to e_1' + e_2} \qquad \frac{e_2 \to e_2'}{n + e_2 \to n + e_2'} \qquad \frac{n_1 + n_2 = n}{n_1 + n_2 \to n}$$

$$\frac{e_1 \to e_1'}{\texttt{let } x = e_1 \texttt{ in } e_2 \to \texttt{let } x = e_1' \texttt{ in } e_2} \qquad \frac{}{\texttt{let } x = v \texttt{ in } e_2 \to e_2\{v/x\}}$$

Règles pour la concaténation $e_1 \hat{} e_2$: idem que $+$.

- La réduction est-elle en appel par nom ou par valeur ?

Noyau OCaml : réduction

$$\frac{e_1 \to e_1'}{e_1 + e_2 \to e_1' + e_2} \qquad \frac{e_2 \to e_2'}{n + e_2 \to n + e_2'} \qquad \frac{n_1 + n_2 = n}{n_1 + n_2 \to n}$$

$$\frac{e_1 \to e_1'}{\texttt{let } x\texttt{=}e_1 \texttt{ in } e_2 \to \texttt{let } x\texttt{=}e_1' \texttt{ in } e_2} \qquad \frac{}{\texttt{let } x\texttt{=}v \texttt{ in } e_2 \to e_2\{v/x\}}$$

Règles pour la concaténation $e_1 \hat{} e_2$: idem que $+$.

- La réduction est-elle en appel par nom ou par valeur ?

  par valeur

- De droite à gauche ou gauche à droite ?

Noyau OCaml : réduction

$$\frac{e_1 \to e_1'}{e_1 + e_2 \to e_1' + e_2} \qquad \frac{e_2 \to e_2'}{n + e_2 \to n + e_2'} \qquad \frac{n_1 + n_2 = n}{n_1 + n_2 \to n}$$

$$\frac{e_1 \to e_1'}{\texttt{let } x\texttt{=}e_1 \texttt{ in } e_2 \to \texttt{let } x\texttt{=}e_1' \texttt{ in } e_2} \qquad \frac{}{\texttt{let } x\texttt{=}v \texttt{ in } e_2 \to e_2\{v/x\}}$$

Règles pour la concaténation $e_1 \hat{} e_2$: idem que $+$.

- La réduction est-elle en appel par nom ou par valeur ?

  par valeur

- De droite à gauche ou gauche à droite ?

  gauche à droite

- Quelles sont les valeurs de ce langage ?

Noyau OCaml : réduction

$$\frac{e_1 \to e_1'}{e_1 + e_2 \to e_1' + e_2} \qquad \frac{e_2 \to e_2'}{n + e_2 \to n + e_2'} \qquad \frac{n_1 + n_2 = n}{n_1 + n_2 \to n}$$

$$\frac{e_1 \to e_1'}{\texttt{let } x\texttt{=}e_1 \texttt{ in } e_2 \to \texttt{let } x\texttt{=}e_1' \texttt{ in } e_2} \qquad \frac{}{\texttt{let } x\texttt{=}v \texttt{ in } e_2 \to e_2\{v/x\}}$$

Règles pour la concaténation $e_1 \hat{} e_2$: idem que $+$.

- La réduction est-elle en appel par nom ou par valeur ?

  par valeur

- De droite à gauche ou gauche à droite ?

  gauche à droite

- Quelles sont les valeurs de ce langage ?

  $v ::= n \mid s \mid x$

Réduction et types

- Formes normales :
  - valeurs 10, “`coucou`”, x, ...
  - Expressions *bloquées* : $10 + $ “`hello`”, $x + 10$, ...

Réduction et types

- Formes normales :
  - valeurs 10, “`coucou`”, x, ...
  - Expressions *bloquées* : $10 + $ “`hello`”, $x + 10$, ...

Definition

Une expression e est *close* si $\mathrm{fv}(e) = \emptyset$.

Remarque : expression close bien typée $\vdash e : \tau$

Réduction et types

"Well-typed programs do not get stuck"

Wright and Felleisen, *A Syntactic Approach to Type Soundness*

Théorème (Progression)

Pour toute expression close e et type τ, si $\vdash e : \tau$, alors e est une valeur, ou il existe e' tel que $e \to e'$.

Réduction et types

"Well-typed programs do not get stuck"
Wright and Felleisen, *A Syntactic Approach to Type Soundness*

Lemme (Formes canoniques)

Pour toute valeur close v :

- *si* $\vdash v : \mathsf{num}$, *alors il existe* n *tel que* $v = n$;
- *si* $\vdash v : \mathsf{str}$, *alors il existe* s *tel que* $v = s$.

Théorème (Progression)

Pour toute expression close e *et type* τ, *si* $\vdash e : \tau$, *alors* e *est une valeur, ou il existe* e' *tel que* $e \longrightarrow e'$.

Réduction et types

"Well-typed programs do not get stuck"
Wright and Felleisen, *A Syntactic Approach to Type Soundness*

Lemme (Formes canoniques)

Pour toute valeur close v :

- *si* $\vdash v : \mathsf{num}$, *alors il existe* n *tel que* $v = n$;
- *si* $\vdash v : \mathsf{str}$, *alors il existe* s *tel que* $v = s$.

Théorème (Progression)

Pour toute expression close e *et type* τ, *si* $\vdash e : \tau$, *alors* e *est une valeur, ou il existe* e' *tel que* $e \to e'$.

Démonstration.

Par induction sur la dérivation de $\vdash e : \tau$. □

Préservation (réduction du sujet)

Théorème (Préservation)

Pour toutes expressions closes e, e *et type* τ, *si* $\vdash e : \tau$ *et* $e \to e'$, *alors* $\vdash e' : \tau$.

Démonstration.

Par induction sur la dérivation de $\vdash e : \tau$. □

Préservation (réduction du sujet)

Théorème (Préservation)

Pour toutes expressions closes e, e *et type* τ, *si* $\vdash e : \tau$ *et* $e \to e'$, *alors* $\vdash e' : \tau$.

Démonstration.

Par induction sur la dérivation de $\vdash e : \tau$. □

- Dans certains langages, type de e' différent
- Preuve par induction sur la dérivation de la réduction possible
- Langage sûr vis-à-vis du typage (*type safe*) : qui vérifie progression et préservation

Terminaison

Définition

Un calcul est *fortement normalisant* si pour tout e, il n'existe pas de suite infinie de réduction $e \to e_1 \to e_2 \to \ldots$

Propriété vrai pour le noyau non typé (calcul trop simple)

Théorème

Le λ-calcul simplement typé (sans stratégie de réduction) est fortement normalisant.

Propriété fausse dans la plupart des langages de programmation (récursion)